SESSION – 10

Editable Template

Table of Contents

Responsive Layout Editing *2*

Introduction to Editable Templates *3*

Templates in AEM **3**

Templates Console **3**

Template Editor **4**

Content Policies **4**

Creating Editable Templates *5*

Creating the Template Folder *5*

Exercise 1: Create an editable template **8**

Task 1: Create a new page rendering structure component 8

Task 2: Create an empty page template type 9

Task 3: Create a development template 11

Task 4: Enable and publish the template 14

Exercise 2: Create a page **17**

Task 1: Create a content root 17

Task 2: Create a site structure 19

Responsive Layout Editing

It is important that websites offer customized views across devices, such as desktops, tablets, and mobile phones. AEM provides the responsive design capability to achieve this customized view.

Responsive Design

In responsive design, the website will respond to fit any screen size through client-side feature detection by using media queries.
Responsive design provides:
Easy reading and access to content

Clear and easy navigation

Minimum of resizing, panning, and scrolling across devices

Making Content Responsive

You can make the content responsive by using the traditional workflow or responsive layout editing.

Traditional Workflow

In a traditional workflow:
A designer mocks different breakpoint

A developer implements the breakpoints for a specific template

An author picks that template and fills out the content

Responsive Layout Editing

In responsive layout editing, the author:
Adds the content to the page

Changes the page layout according to the device

Responsive Layout

In AEM, you can add the responsive layout to pages by using the following combination of mechanisms:
Layout container component: Provides a grid-paragraph system to add and position components within a responsive grid.

Introduction to Editable Templates

In Adobe Experience Manager (AEM), template authors use templates for creating pages. AEM provides both static templates and editable templates. Authors can create and configure the editable templates without a development project or iteration. However, to enable this capability in AEM, developers must set up the required environment and create client libraries and components to be used on the page.

Objectives

After completing this module, you will be able to:

- Explain templates in AEM
- Create editable templates
- Create pages from editable templates

Templates in AEM

Content authors can easily create pages and content from the AEM Sites console.

The AEM template defines the content structure with JCR nodes and properties. Templates use the node type **cq:Template** as the root node.

Types of Templates

The following table describes both static and editable templates:

Static Templates	Editable Templates
Are developed and configured by developers	Are created and edited by authors
Have the same structure as the page	Help define the structure, initial content, and content policies for pages
Are copied to create a new page and do not have a dynamic connection with the page	Maintain a dynamic connection between the template and pages
Use the Design mode to persist design properties	Use content policies (edited from the template editor) to persist the design properties
Are stored under /apps	Are stored under /conf

Templates Console

The Templates console enables template authors or power authors to:

- Create a new template
- Edit the template by using the template editor

- Manage the template life cycle

  You can access the Templates console from the Tools > General section.

Template Editor

The template editor enables template authors to:

- Add the available components to the template and position them on a responsive grid
- Preconfigure components
- Define the components that you can edit on the resultant pages (created from the template)
- Compose templates out of available components
- Manage the lifecycle of templates

The template editor has the following modes:

Structure: The structure enables template authors to define the components, such as header, footer, and layout container for a page. Template authors can add a paragraph system to the template, which helps page authors to add and remove the components on the resultant pages. When components are locked, page authors cannot edit the content. In Structure mode, any component that is the parent of an unlocked component cannot be moved, cut, or deleted.

Initial Content: Power authors can define the content that needs to be included in all pages, by default. When a component is unlocked, template authors can define the initial content that will be copied to the resultant page(s) created from the template. Page authors can edit these unlocked components on the resultant page(s). In the initial content mode (and on the resultant pages), you can delete any unlocked components with an accessible parent, such as components within a layout container.

Layout: The Layout enables power authors to predefine the template layout for the required device formats.

Content Policies

You can define content policies for templates and components from the template editor.

The content policies:

- Connect the predefined page policies to a page. These page policies define various design configurations.
- Enable you to assign a predefined design configuration to selected components. This helps preconfigure a component's behaviour on the resultant page.
- Enable you to add the allowed components and default components to the template.
- Define the number of columns (responsive settings) in the template.

Creating Editable Templates

The steps to create an editable template are:

1. Create a context-aware configuration folder and enable editable templates.
2. Create a template type for your template.
3. Create a new template from the template type.
4. Define additional properties for the template, if required.
5. Edit the template to define the structure, initial content, layout, and content policies.
6. Enable the template.
7. Make the template available for the required page or the branch of your website.
8. Publish the template so that it is available on the publish environment.

Creating the Template Folder

To organize your templates, you can use the following folders:

- **global:** In a standard AEM author instance, the global folder exists in the Templates console. This folder contains the default templates and acts as a fallback if no policies and/or template-types are found in the current folder. You can add your default templates to global folder or create a new folder (recommended).
- **site-specific:** The site-specific folders help organize templates specific to your site.

Best Practice: Create a new folder to save your customized templates instead of using the global folder. A user with admin rights can create folders.

Creating a Template Folder from the Configuration Browser Console

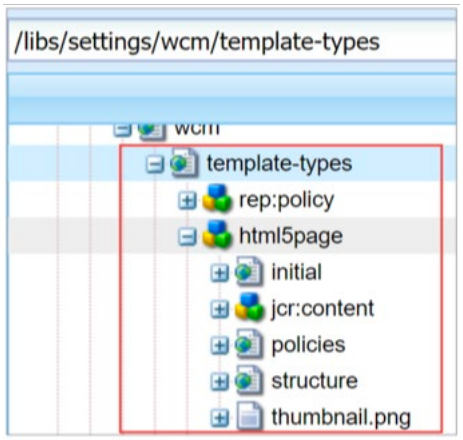
You can access the **Configuration Browser** console from the **Tools** console. You can create a multi-tenant environment where tenants can have different editable templates, configurations, cloud services, and data models.

Creating a Template Type

When creating a new template, you must specify a template type. The template type is copied to create the template, and the structure and initial content of the selected template type is used to create the new template. After the template type is copied, the only connection between the template and the template type is a static reference for information purposes.
Template types help you define the:

- Resource type of the page component
- Policy of the root node, which defines the components allowed in the template editor

The out-of-the box template types are stored under **/libs/settings/wcm/template-types,** as shown:



Template Definitions

The definitions for editable templates are stored under user-defined folders (recommended) or alternatively in the global folder. For example:
/conf/<my-folder>/settings/wcm/templates

/conf/<my-folder-01>/<my-folder-02>/settings/wcm/templates

The root node of the template is of type **cq:Template** with the following skeleton structure:

```
<template-name>
  initial
    jcr:content
      root
        <component>
        ...
        <component>
  jcr:content
    @property status
  policies
    jcr:content
      root
        @property cq:policy
        <component>
          @property cq:policy
        ...
        <component>
          @property cq:policy
```

The main elements are:

- <template-name>
- jcr:content
- structure
- initial
- policies
- thumbnail.png

jcr:content
The jcr:content node holds the following properties of the template:

- Name: jcr:title
- Name: status
    - Type: String
    - Value: draft, enabled, or disabled

structure
The structure node defines the structure of the resultant page. The structure of the template is merged with the initial content (/initial) when creating a new page. Any changes to the structure will be reflected in any pages created with the template.
The root (structure/jcr:content/root) node defines the list of components that will be available in the resulting page. The components defined in the template structure cannot be moved or deleted from any resultant pages. After a component is unlocked, the editable property is set to true. After a component that already contains content is unlocked, the content is moved to the initial branch. The **cq:responsive** node holds the definitions for the responsive layout.

initial
The initial node defines the initial content that a new page will have upon creation. The initial node contains a **jcr:content** node that is copied to any new pages and is merged with the structure (/structure) when a new page is created. Any existing pages will not be updated if the initial content is changed after creation. The root node holds a list of components to define what will be available in the resulting page. If content is added to a component in Structure mode and that the component is subsequently unlocked (or vice versa), the content is used as initial content.

policies
The content (or design) policies define the design properties of a component. For example, the components available or the minimum/maximum dimensions. These are applicable to the template (and pages created with the template). Content policies can be created and selected in the template editor. The property `cq:policy`, on the **/conf/<your-folder>/settings/wcm/templates/<your-template>/ policies/jcr:content/root** root node, provides a relative reference to the content policy for the page's paragraph system. The property **cq:policy**, on the component-explicit nodes under root, provides links to the policies for the individual components. The actual policy definitions are stored under **/conf/<your-folder>/settings/wcm/policies/wcm/foundation/components**.

Note: The paths of policy definitions depend on the path of the component. cq:policy holds a relative reference to the configuration itself.

Layout
When editing a template, you can define the layout. The layout uses the standard responsive layout that can be configured.

Page Policies

Page policies help define the content policy for the page (main parsys) in either the template or the resultant pages.

Editing the Template

After creating a template, you can edit the template from the template editor. The changes made to the template will have impact on the existing pages depending on the template editor modes. For example, if you make changes to the structure of the template, it will apply immediately to all pages created from the template immediately. It is also possible to define the initial content for a template, which will be copied over to newly created pages.

Enabling the Template

Before you use a template, you must enable it:

- From the **Templates** console

OR

- By setting the status property on the jcr:content node For example, you can change the property of **conf/<your-folder>/settings/wcm/templates/<your-template>/jcr:content**
- Define the following properties: Name: status
    - Type: String
    - Value: enabled

To ensure you can access the template, you must:

1. Define the **Allowed Template path(s)** on the **Page Properties** of the appropriate page or root of the page of a subbranch.
2. Set the property **cq:allowedTemplates** on the jcr:content node of the required branch. For example, **/conf/<your-folder>/settings/wcm/templates/.***.

Publishing the Template

As the template is referenced when a page is rendered, the (fully configured) template must be published from the **Templates** console to make it available on the publish environment.

Exercise 1: Create an editable template

In this exercise, you will perform the following tasks:

1. Create an empty template type
2. Create a development template
3. Enable and publish the template

Task 1: Create a new page rendering structure component

Before we create editable template, we need to create a new page/structure component which will inherit from the core component's page component.

1. Right-dick /apps/training/components/structure and select Create...> Create Component.
2. Fill in the following values:
    - **Label**: page
    - **Title**: WE.TRAIN page Component

- **Description**: WE.TRAIN page-rendering Component
- **SuperType**: core/wcm/components/page/v2/page

3. Click Next.
4. Click OK, and save your changes.
5. Right-dick on page.jsp to rename it to page.html. Click on Save All.
6. Now your page component is ready to work with editable templates.

Task 2: Create an empty page template type

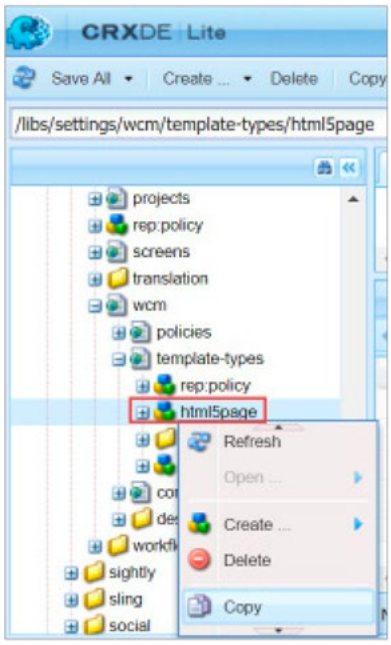
Before you create a template type, check the template types that are available out of the box.

1. Navigate to **Adobe Experience Manager** > **Tools** > **Templates** if you are not in the **Templates** console already.
2. Click the **We.Train** folder and click **Create** from the actions bar. The **Create Template page** opens. Notice that two template-types are available now. These are the default template-types and if you used one of these template-types, you would use the default components. You must create your own components and design so that you can create your own template-type based on the HTML5 Page type.
3. Click **Cancel** to close the **Create Template** page.

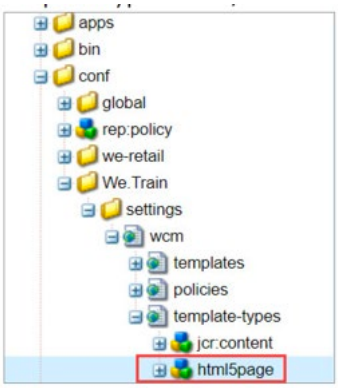
Template-types connect to a base page component. The page components are responsible for ensuring that global areas of the site are consistent. This includes loading of global CSS and Javascript as well as the inclusion of code that will ensure the page can be edited by using AEM authoring tools. You will use the training/components/structure/page component you created earlier.

To create a custom template type:

4. Click Adobe Experience Manager from the header bar. The Tools console opens.
5. Click CRXDE Lite. The CRXDE Lite page opens.
6. Navigate to the /libs/settings/wcm/template-types/html5page node.
7. Right-click the html5page node and click Copy from the drop-down menu, as shown:



8. Navigate to the /conf/we-train/settings/wcm/template-types folder.
9. Right-click the template-types folder and click Paste. The html5page node is added to the template-types folder, as shown:



10. Click Save All from the actions bar.
11. Right-click the html5page node and click Rename.
12. Rename the node to empty-page.
13. Select the/jcr:content node under empty-page.
14. Modify the properties under the **Properties** tab:
    a. **jcr:title**: **We.Train Empty Page**
    b. **jcr:description**: **Empty Template for a We.Train page**
15. Click Save All from the actions bar.
16. Select the empty-page/initial/jcr:content node.
17. Modify the property under the Properties tab:

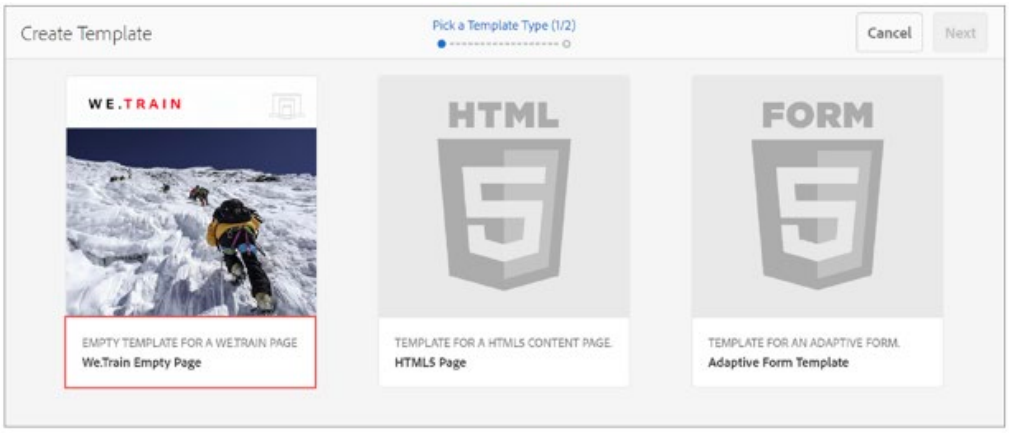
    a. **sling:resourceType**: training/components/structure/page

18. Click Save All from the actions bar.
19. Select the empty-page/structure/jcr:content node.
20. Modify the property under the Properties tab:

    a**. sling:resourceType: training/components/structure/page**
21. Click Save All from the actions bar.
22. Navigate to empty-page/policies/jcr:content, select the root node, right-click it, and click Delete to delete the node. Now, you can start with an empty root layout container.
23. Click Save All from the actions bar.
24. Navigate to the empty-page/thumbnail.png/jcr:content node.
25. On the Properties tab, double-click the jcr:data. The Edit jcr:data dialog box opens.
26. Click Browse. The Open dialog box opens.
27. Navigate to the Creating Editable Templates and Pages in Adobe Experience Manager exercise folder on your file system, select the We.Train-thumbnail.png image, and click Open.
28. Click OK. The thumbnail image is added to template-type.
29. Click Save All to save changes.

To test whether the template type is available for creating editable templates:

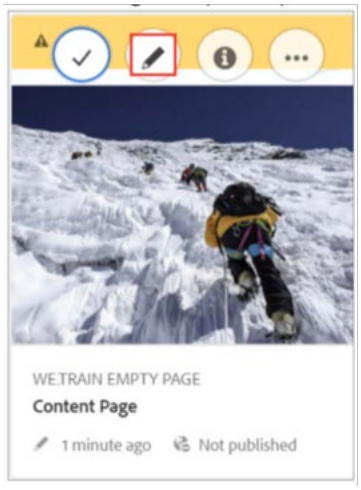
30. Go to AEM start page and from there Click the Tools icon and click Templates. The Templates console opens.
31. Click the We.Train folder and click Create from the actions bar. You should now see the We.Train Empty Page template type.



Task 3: Create a development template

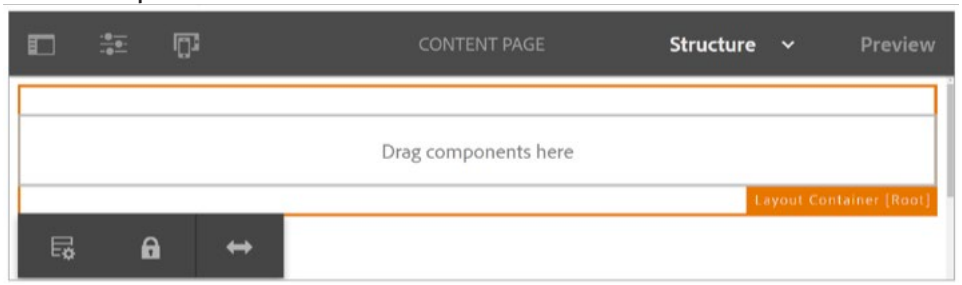
After creating the template-type, you must create a development template. The development template enables you to use the template authoring environment and to setup initial template configurations. After the setup is ready in the development template, you will copy it back to the template type.
To create a development template:

1. If you are not already in the Create Template wizard, navigate to Adobe Experience Manager > Tools > Templates > We.Train folder.
2. Click Create from the actions bar. The Create Template wizard opens.
3. Select the We.Train Empty Page template, and click Next.
4. In the Template Title field, enter Content Page and click Create. The Success dialog box opens.
5. Click Done in the Success dialog box.
6. Hover the cursor over the Content Page template, and click the Edit icon, as shown. The Content Page template opens in a new browser tab.



At this point, you will see the default page.html script that you installed in a previous task. To use the editable templates, you need to inherit the default script from the Core Page component. To do that:

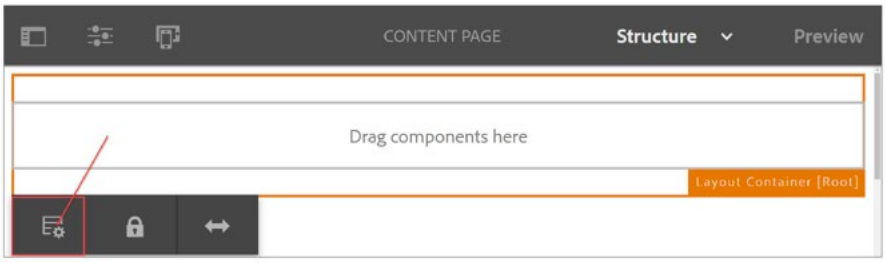
7. Click http://localhost:4502/crx/de/index.jsp. The **CRXDE Lite** page opens.
8. Navigate to the **/apps/training/components/structure/page** node.
9. Right-click **page.html** and click **Delete**. The page is deleted.
10. Click **Save All** from the actions bar.
11. In your browser, navigate to the tab where the Content Page template is open, and refresh the page. Notice that the page.html script does not appear and the **Layout Container [root]** is now visible, as shown. The Layout Container [root] helps configure the components that can be added to the template.



Note: The root layout container is visible to template authors only when editing the template. This container will not be available on the page.

You must now add a few structure components to the root layout container, so that you can start building the template.

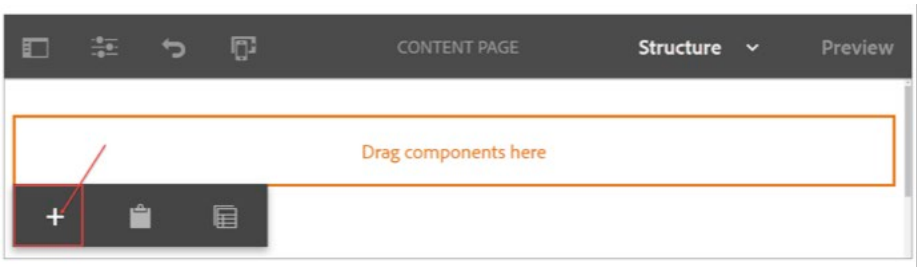
12. Select the **Layout Container [root]** and click the **Policy** icon, as shown. The **Layout Container** wizard opens.



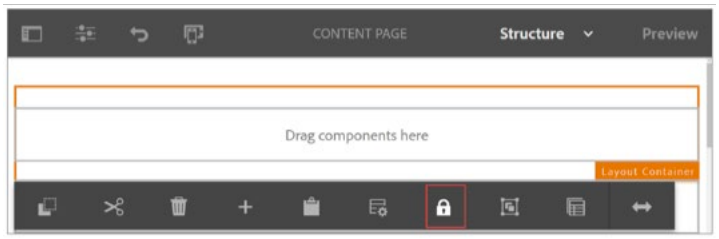
13. In the Policy section, in the Policy Title field, enter We.Train Structure Policy.
14. In the Properties section, ensure you are in the Allowed Components tab, and expand the General group. The list of components available in the group is displayed.
15. Select the Content Fragment, Experience Fragment, and Layout Container checkboxes.
16. Click Done (the checkmark icon) on the upper-right. The selected components are added to the policy.

Now that you have structure components in the template, you will add a layout container that will be used in the resulting pages. This layout container will contain approved content components that authors can drag and drop onto a page.

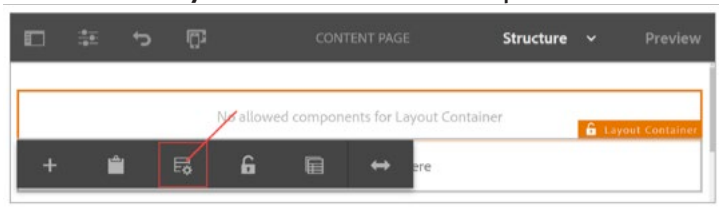
17. Refresh the page.
18. Ensure you are in the Structure mode and click within the Drag components here layout.
19. Click the Insert component icon, as shown. The Insert New Component dialog box opens.



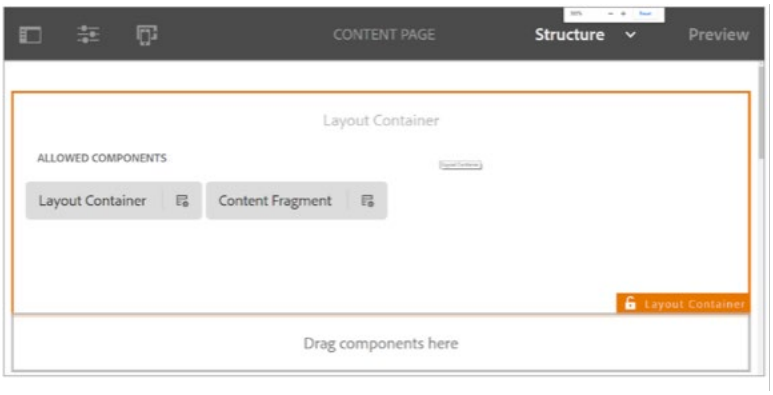
20. Select Layout Container from the list of components. The component is added to the template.
21. Select the inner Layout Container (not the root Layout Container) and click the Unlock Structure Component icon from the component toolbar, as shown. This enables you to add components to the container.



22. Select the inner **Layout Container** and click the **Policy** icon from the component toolbar, as shown. The **Layout Container** wizard opens.



23. In the Policy section, in the Policy Title field, enter We.Train Content Policy.
24. In the Properties section, ensure you are in the Allowed Components tab, and expand the General group. The list of components available in the group is displayed.
25. Select the Content Fragment, Experience Fragment, and Layout Container checkboxes.
26. Click Done (the checkmark icon) on the upper-right. The selected components are added to the policy.

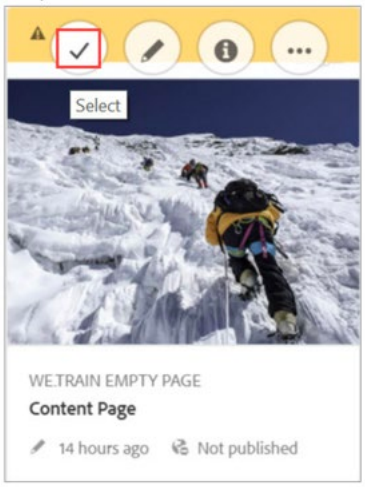


Task 4: Enable and publish the template

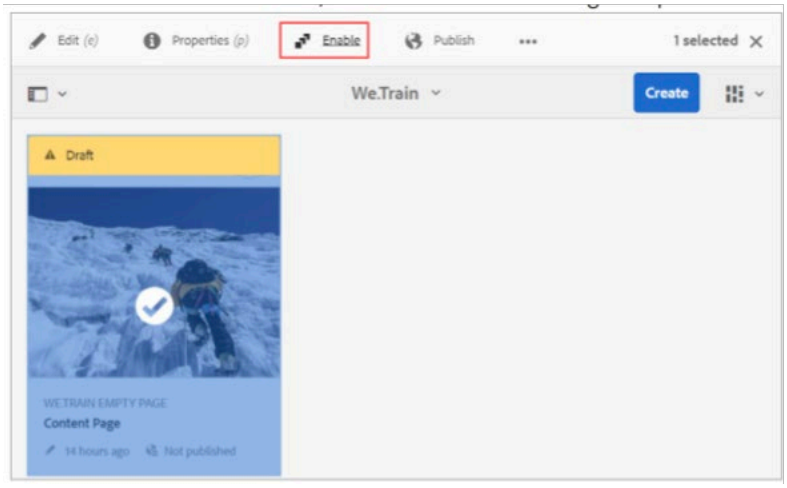
In this exercise, you will first enable the template so that it is available for authors to create pages on the author instance. Later, publish the template to make it available on the publish instance.
To enable the template:

1. In the AEM author instance, navigate to Adobe Experience Manager > Tools > Templates. The Templates console opens.
2. Click the We.Train folder. The Content page template you created earlier is displayed.

3. Hover the cursor over the Content Page template and click the Select icon, as shown. The template is selected.



4. Click Enable from the actions bar, as shown. The Enable dialog box opens.



5. Click Enable. Notice that the status of Content Page changes to Enabled.
6. Ensure the Content Page template is selected and click Publish from the actions bar. The Publish dialog box opens.
7. Ensure all checkboxes are selected and click Publish. The message INFO The page has been published appears.

Creating Pages from Editable Templates

Before creating pages through the UI, a content root needs to be set up for your site. The content root will define the allowed templates for the site and is used to set other global configurations. By

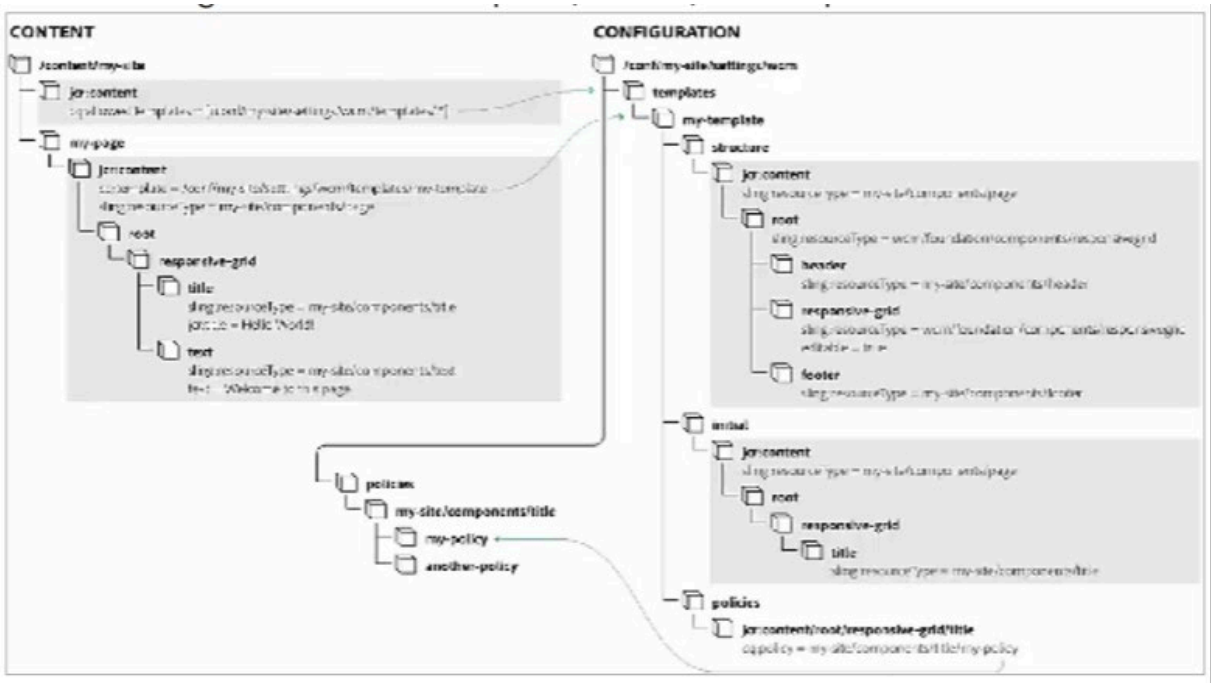
convention, the content root is not intended to be the home page for the site and instead will redirect to the true home page.

Creating Pages

In the **Sites** console, you can create the pages of your website by using a template. The pages created from editable templates:
Are created with a subtree that is merged from the structure and initial in the template

Have references to the information held in the template and the template type. This is achieved with a jcr:content node with the following properties: cq:template: Provides the dynamic reference to the actual template, and enables changes to the template to be reflected on the actual pages.
cq:templateType: Provides a reference to the template type.

The below diagram shows how templates, content, and components interrelate:



In the above diagram:
Controller: The resultant page that references the template (**/content/<my-site>/<my-page>)** The content controls the entire process. According to the definitions, it accesses the appropriate template and components.

Configuration: The template and related content policies define the page configuration (/**conf/<my-folder>/settings/wcm/templates/<my-template>**)
Model: The OSGI bundles implement the functionality.
View: On both author environment and publish environment, the content is rendered by components (/apps/<my-site>/components).

When rendering a page:
The **cq:template** property of its **jcr:content** node is referenced to access the template that corresponds to that page.

The page component will merge the **structure/jcr:content** tree of the template with the **jcr:content** tree of the page.

The page component will allow the author to edit only the nodes of the template structure that are flagged as editable.

The relative path of the component is taken from the jcr:content node when rendering a component on a page.

The **cq:policy** property of the component: Points to the actual content policy (holds the design configuration for that component).
Helps you have multiple templates that re-use the same content policy configurations.

Exercise 2: Create a page

In this exercise, you will create a content root and site structure for the website. The content root will help you use the Content Page template when creating pages.
This exercise includes the following tasks:

1. Create a content root
2. Create a site structure

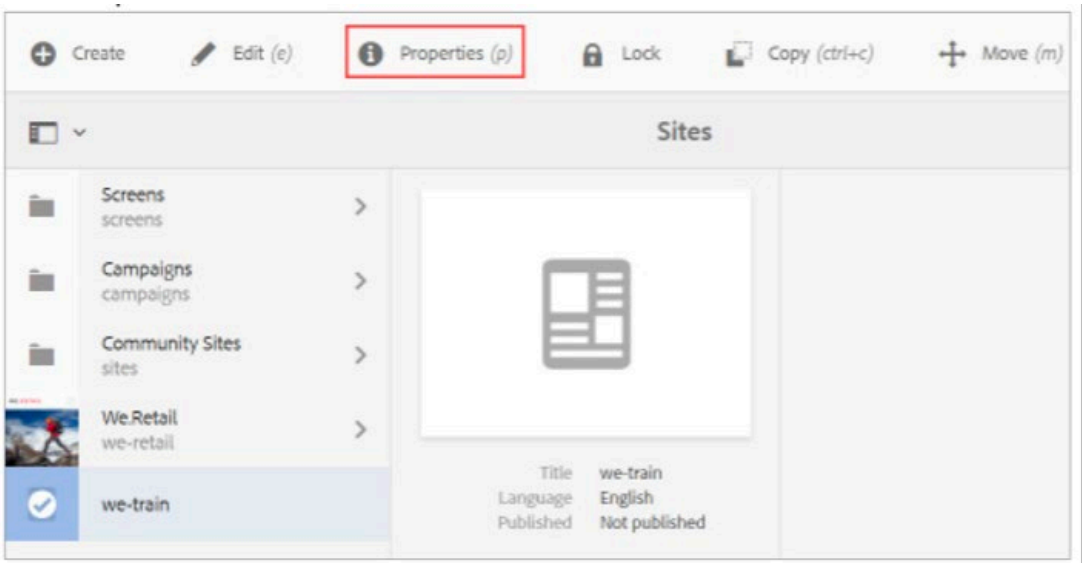
Task 1: Create a content root

1. Since we will be using same name "**we-train**" for our content root node for the website which we will create using editable templates. Rename the content root for the website which we created using static template to "we-train-old". In this way we will keep the work which we have done during static template exercises.
2. Click http://localhost:4502/crx/de/index.jsp The CRXDE Lite page opens.
3. Right-click the /content folder and click Create > Create Node. The Create Node dialog box opens.
4. In the Name field, enter we-train.
5. From the Type drop-down menu, select cq:Page.
6. Click OK. The node is created
7. Click Save All.
8. Right-click the we-train node, and click Create > Create Node. The Create Node dialog box opens.
9. In the Name field, enter jcr:content.
10. From the Type drop-down menu, select cq:PageContent.
11. Click OK. The node is created
12. Click Save All.

13. Select the jcr:content node that is below the we-train node.
14. On the Properties tab, add the following property:

Name	Type	Value
sling:resourceType	String	training/components/structure/page

15. Click Add to add the property.
16. Click Save All to save the changes.
17. In your AEM author instance, click Adobe Experience Manager.
18. Click the Navigation icon on the left panel and click Sites. The Sites console opens.
19. Click the thumbnail beside the we-train page and click Properties as shown. The Properties wizard opens.



20. Ensure you are in the Basic tab and enter We.Train in the Title field.
21. On the Advanced tab:
    a. In the Redirect field, enter /content/we-train/en.
    b. Scroll down and click Add under Allowed Templates.
    c. Enter /conf/we-train/settings/wcm/templates/.* in the Allowed Templates field.
22. Click Save & Close. The form has been submitted successfully message appears.

To view the properties that you added to content root in JCR:

23. In CRXDE lite, navigate to the /content/we-train/jcr:content node and observe the properties that you just added to the content root.

Task 2: Create a site structure

1. You have enabled and published the Content Page template and created a content root. Next, you can create your site structure.
2. In this exercise, you will create the following website structure:

--> We.Train
--------> English
----------> About Us
----------> Experiences
----------> Equipment
----------> Activities
----------> Hiking
----------> Biking
----------> Running
----------> Skiing
----------> Climbing

Now you know how to create pages. So prepare a content structure as shown in below image:

